

Transfer Learning

Matthew Engelhard

Last Time

- Key importance of computer vision in medicine
- Example CV problems
- How do CNNs work?
 - Hierarchical feature extraction
 - Translation invariance
- The convolution operation
- Early CNN architectures

Today

- Transfer learning overview
- Parameter efficient models and fine-tuning
- Residual connections
- Multiple-instance learning
- Segmentation architectures
- An initial look at zero-shot prediction

How do we build our CNN?

Copy, modify, fine-tune. We will focus on whole-image classification.

Step 1. Download a trained image classification model.

Pre-trained Models

Neural nets work best when they have many parameters, making them powerful function approximators. However, this means they must be trained on very large datasets. Because training models from scratch can be a very computationally intensive process requiring days or even weeks, we provide various pre-trained models, as listed below. These CNNs have been trained on the [ILSVRC-2012-CLS](#) image classification dataset.

In the table below, we list each model, the corresponding TensorFlow model file, the link to the model checkpoint, and the top 1 and top 5 accuracy (on the imagenet test set). Note that the VGG and ResNet V1 parameters have been converted from their original caffe formats ([here](#) and [here](#)), whereas the Inception and ResNet V2 parameters have been trained internally at Google. Also be aware that these accuracies were computed by evaluating using a single image crop. Some academic papers report higher accuracy by using multiple crops at multiple scales.

Model	TF-Slim File	Checkpoint	Top-1 Accuracy	Top-5 Accuracy
Inception V1	Code	inception_v1_2016_08_28.tar.gz	69.8	89.6
Inception V2	Code	inception_v2_2016_08_28.tar.gz	73.9	
Inception V3	Code	inception_v3_2016_08_28.tar.gz	78.0	
Inception V4	Code	inception_v4_2016_09_09.tar.gz	80.2	

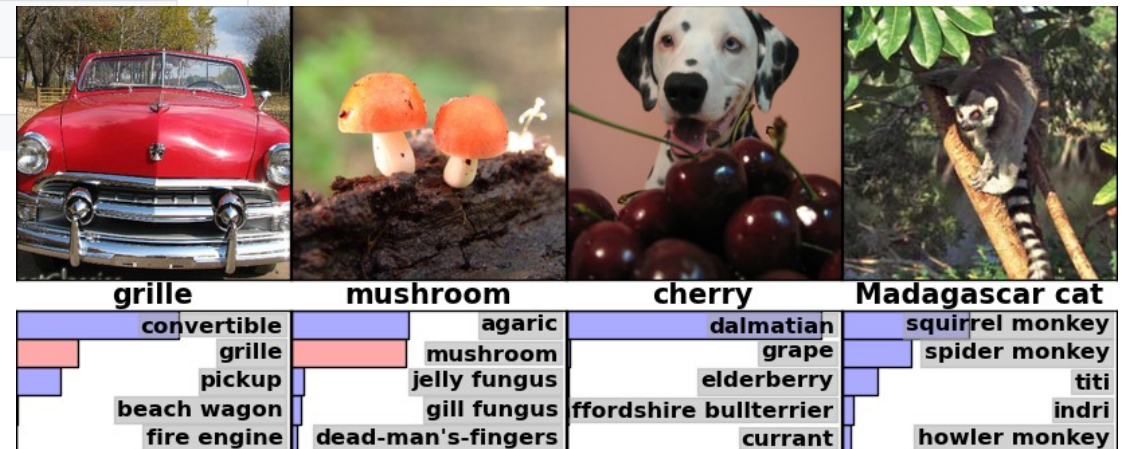
<https://github.com/tensorflow/models/tree/master/research/slim#Pretrained>

Python Code:

Defines the model architecture

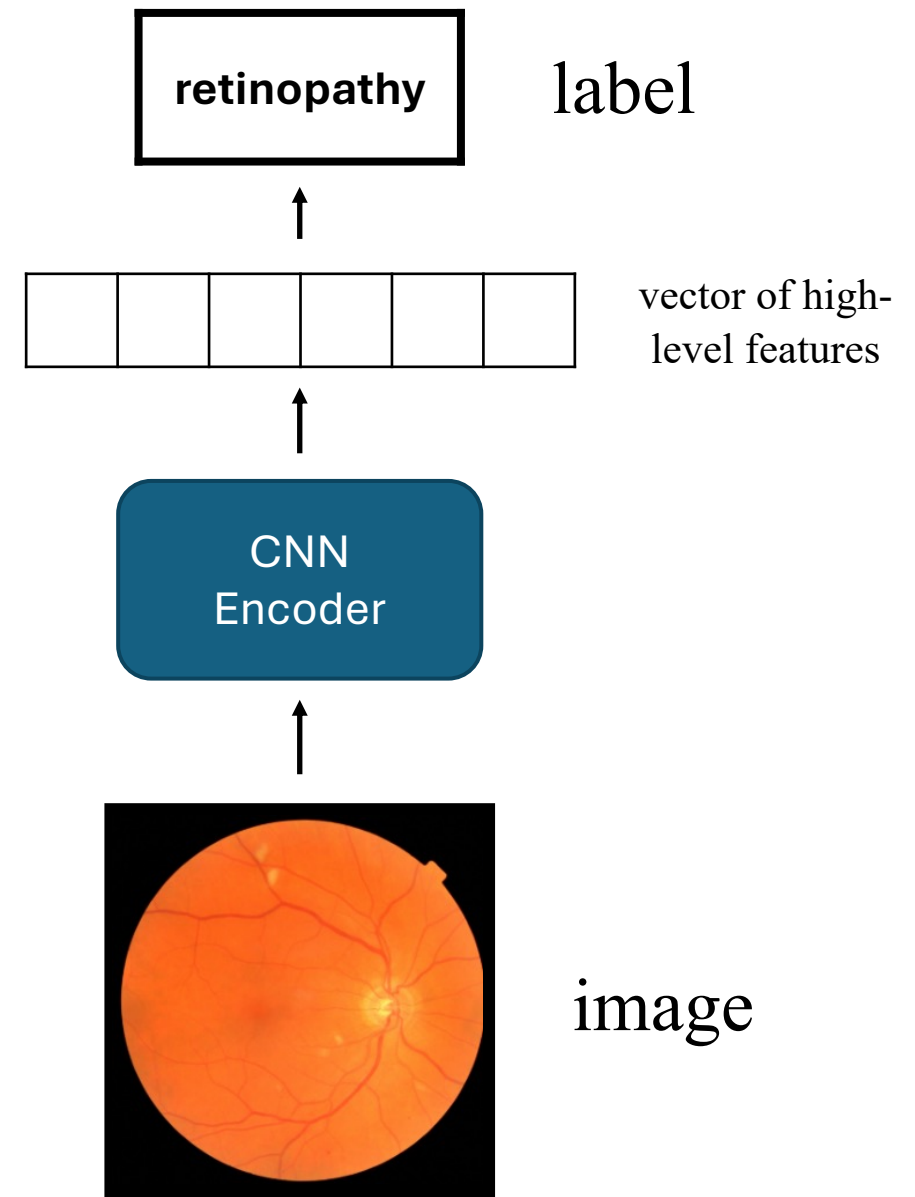
Checkpoint File:

Contains parameters of the trained model



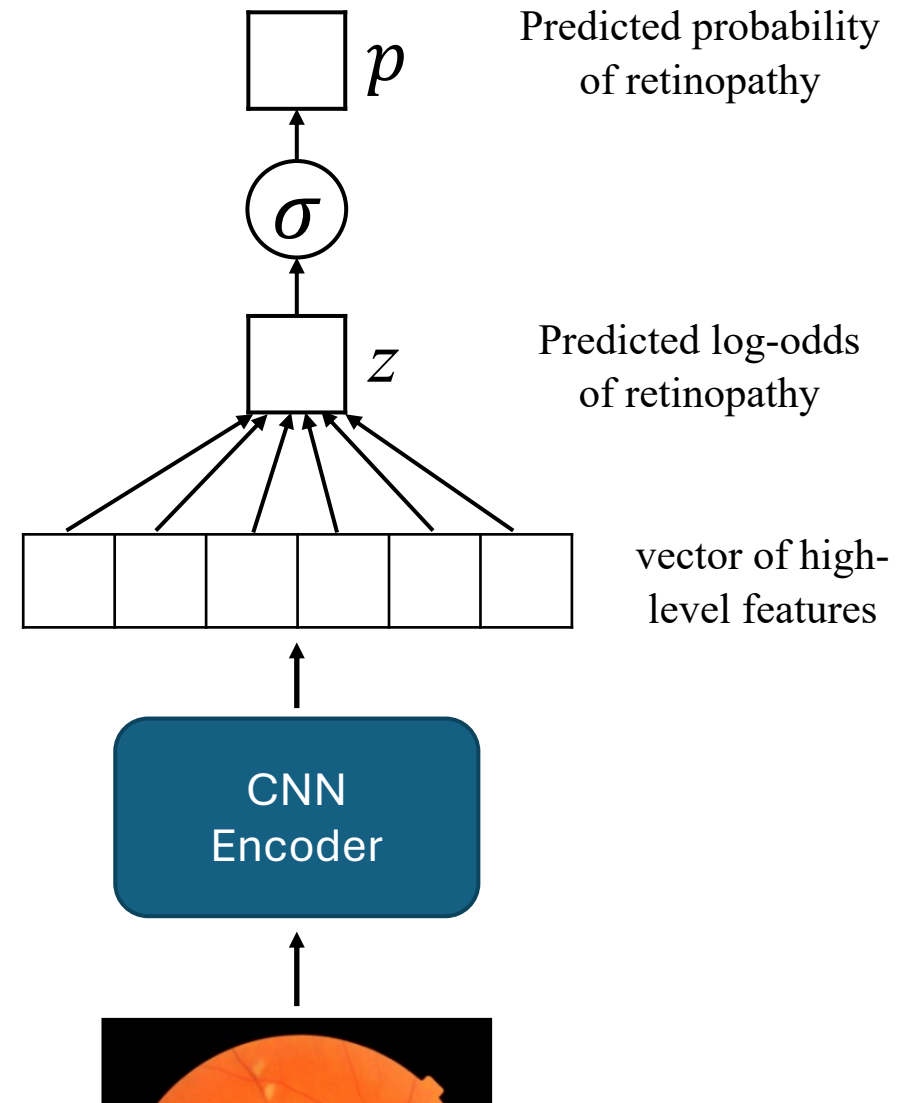
We can divide this model into two pieces:

1. A CNN *image encoder* that converts the raw image to a vector of high-level motifs / features.
 2. A *final layer, or prediction head* – this is a logistic regression model – that makes predictions about the label from these high-level features.
- We will reuse the encoder but replace the prediction head, since it is specific to the previous (non-medical) task.



(Enhance!) Inspecting the prediction head:

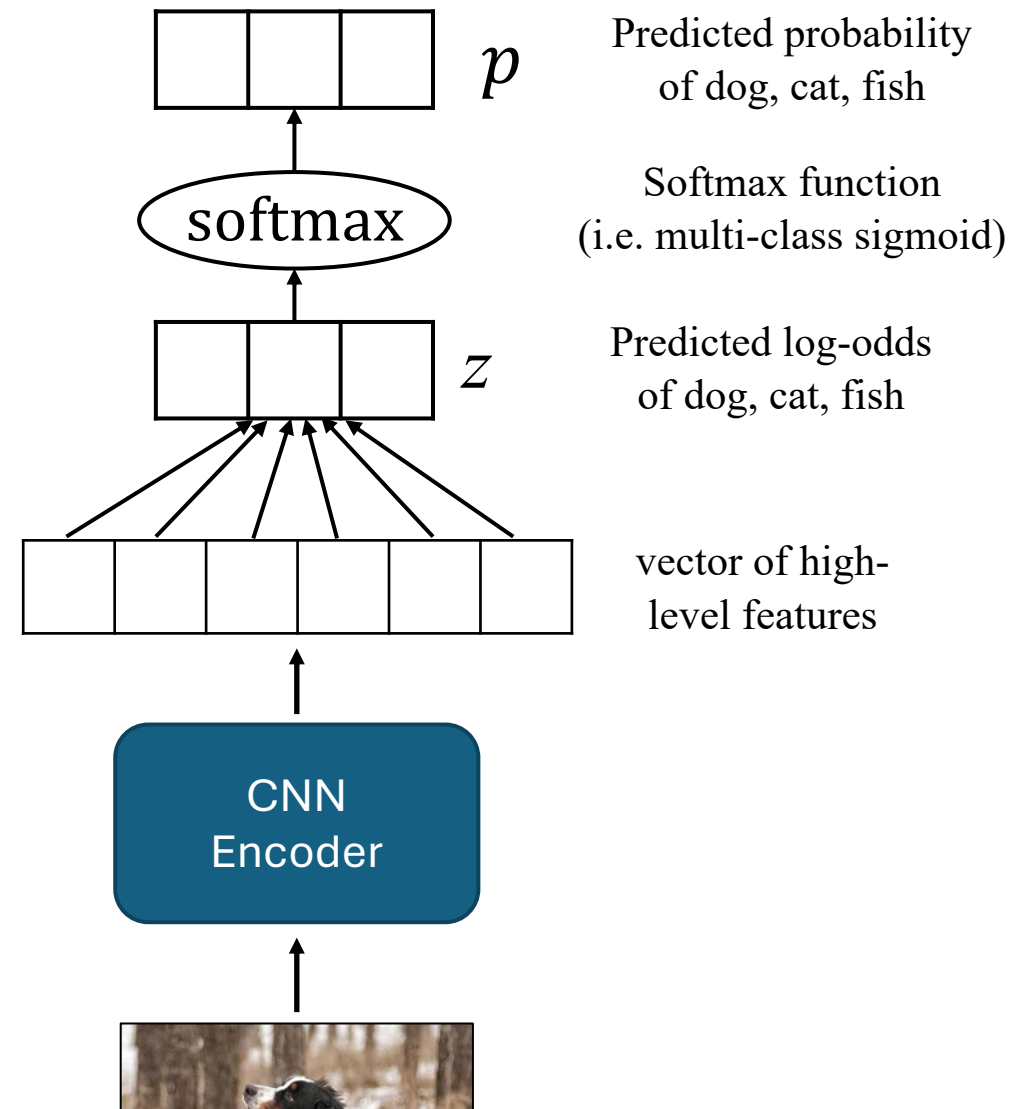
In this example, our prediction head is simple logistic regression on the features extracted by the CNN encoder.



(Enhance!) Inspecting the prediction head:

In this example, our prediction head is simple logistic regression on the features extracted by the CNN encoder.

Typically, we start with a multi-class (rather than binary) prediction head.

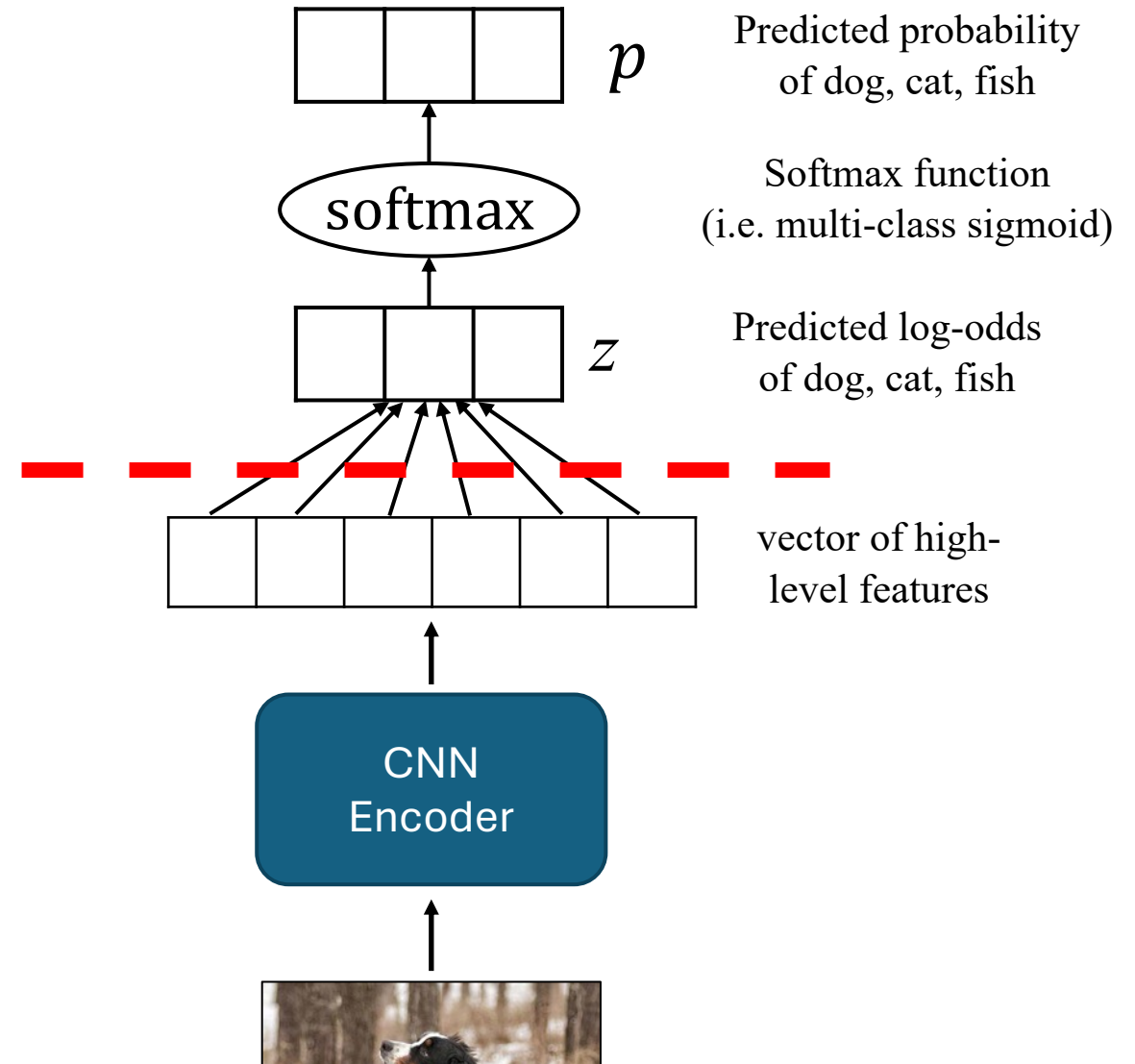


Step 2: Replace the prediction head.

In this example, our prediction head is simple logistic regression on the features extracted by the CNN encoder.

Typically, we start with a multi-class (rather than binary) prediction head.

We want to make predictions about a new set of labels, so we cut off the prediction head...

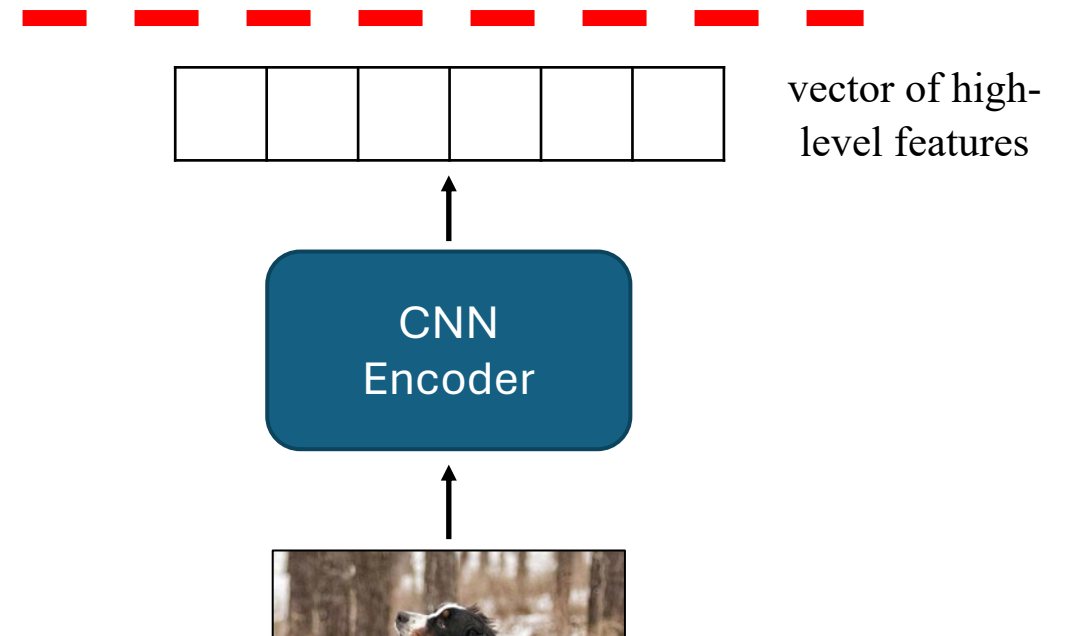


Step 2: Replace the prediction head.

In this example, our prediction head is simple logistic regression on the features extracted by the CNN encoder.

Typically, we start with a multi-class (rather than binary) prediction head.

We want to make predictions about a new set of labels, so we cut off the prediction head...



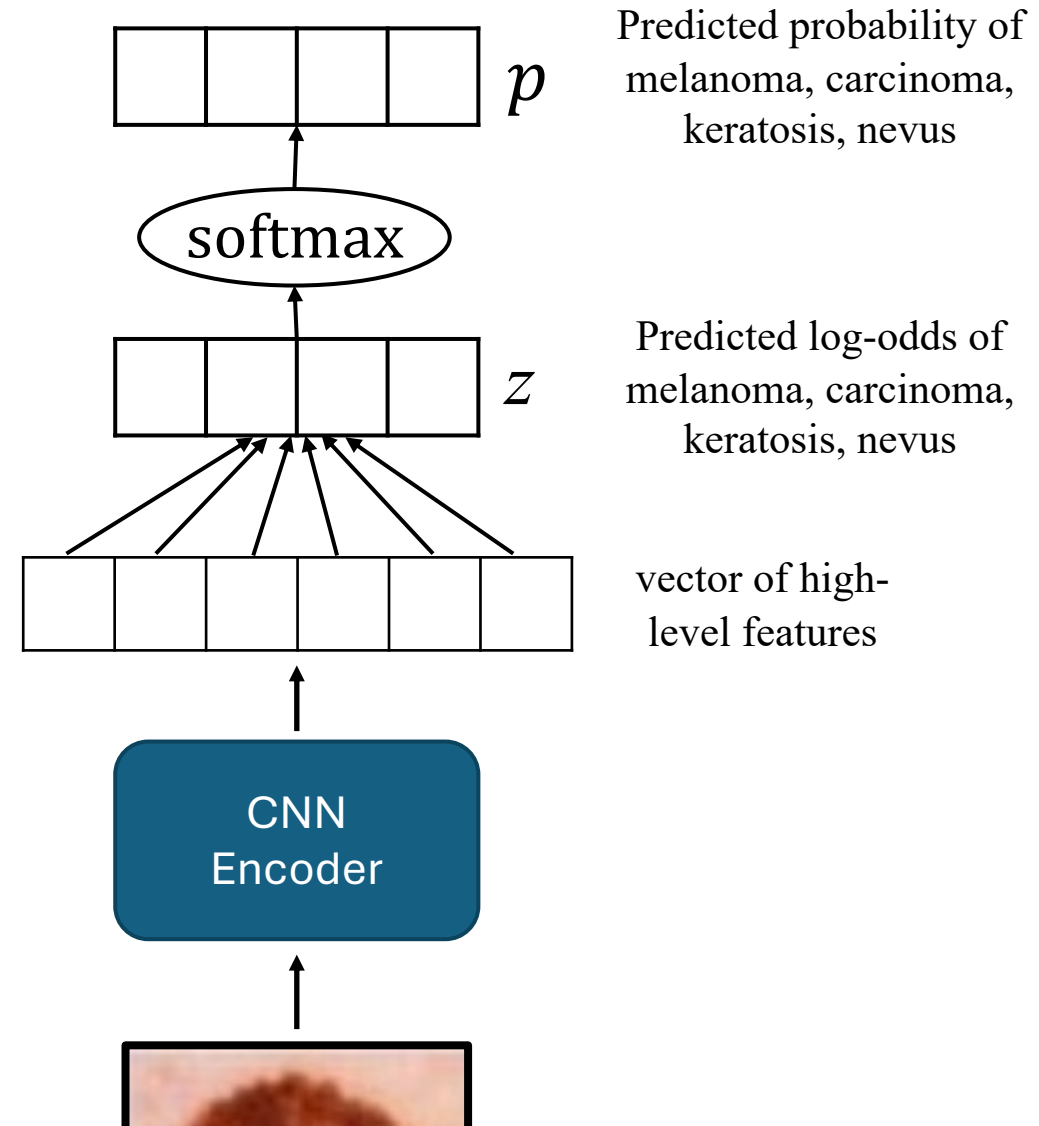
Step 2: Replace the prediction head.

In this example, our prediction head is simple logistic regression on the features extracted by the CNN encoder.

Typically, we start with a multi-class (rather than binary) prediction head.

We want to make predictions about a new set of labels, so we cut off the prediction head...

...and replace it with a new one appropriate for our new prediction task.



Step 3: Train it on your data.

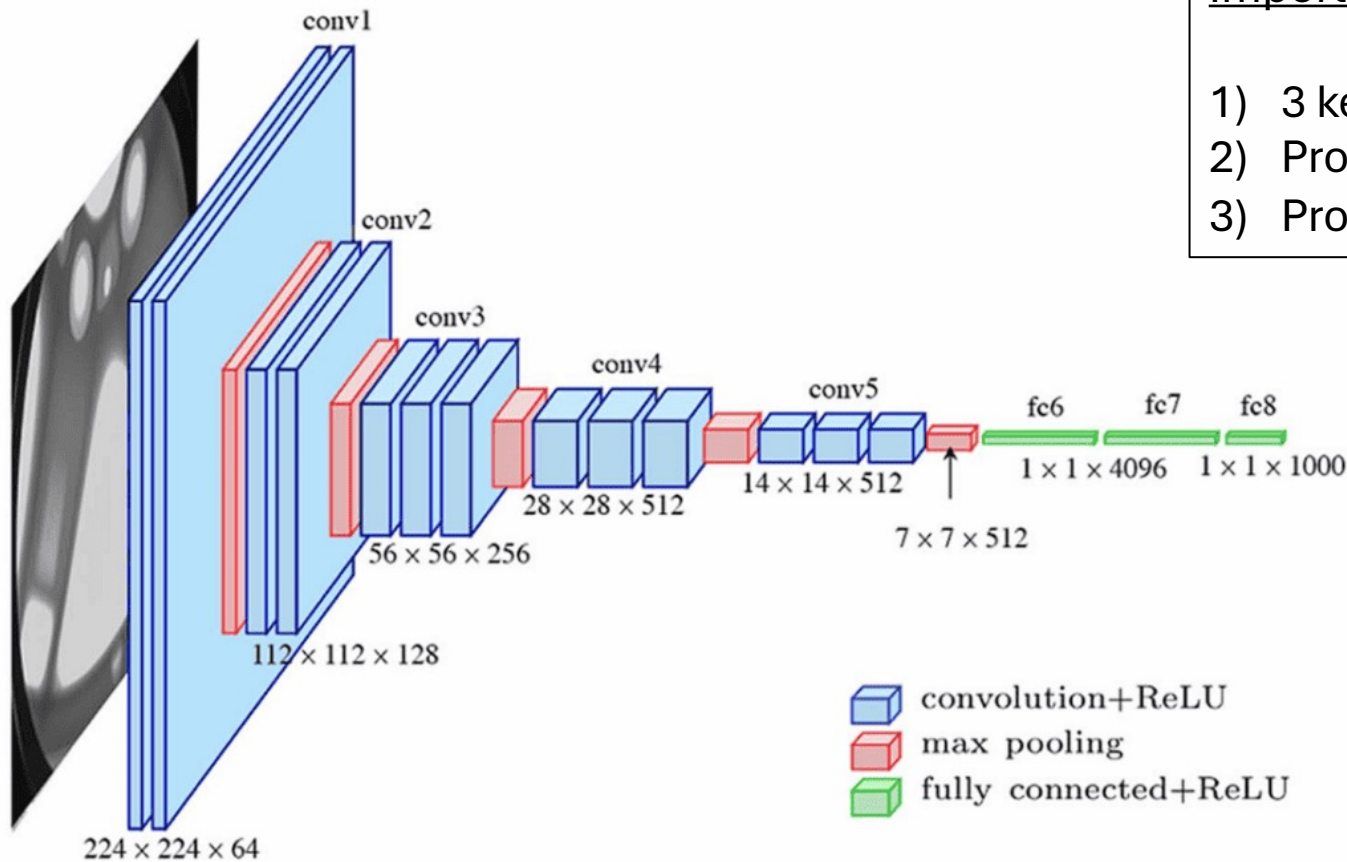
- Finally, you train the model with your dataset of medical images.
- You train the same way you'd train any other model.*
- The prediction head is new, so parameters in this layer are learned entirely from your data.
- The CNN encoder is not new, so we say parameters in the encoder are *fine-tuned* to your data.
- You'll need a big dataset (e.g. 1k), but since you started with a pre-trained model, not as big as if you trained from scratch (e.g. 1M).
- This process (steps 1-3) is called *transfer learning*.

*with a few caveats

Some notes on fine-tuning:

- 1) fine-tuning a pre-trained model tends to be **at least as good as learning from scratch**
(empirical result)
- 2) freeze early layers and fine-tune later layers
more data → fine-tune more layers
- 3) best tuning depth depends on the application, and should be explored as a hyperparameter

Which features to use depends on the architecture



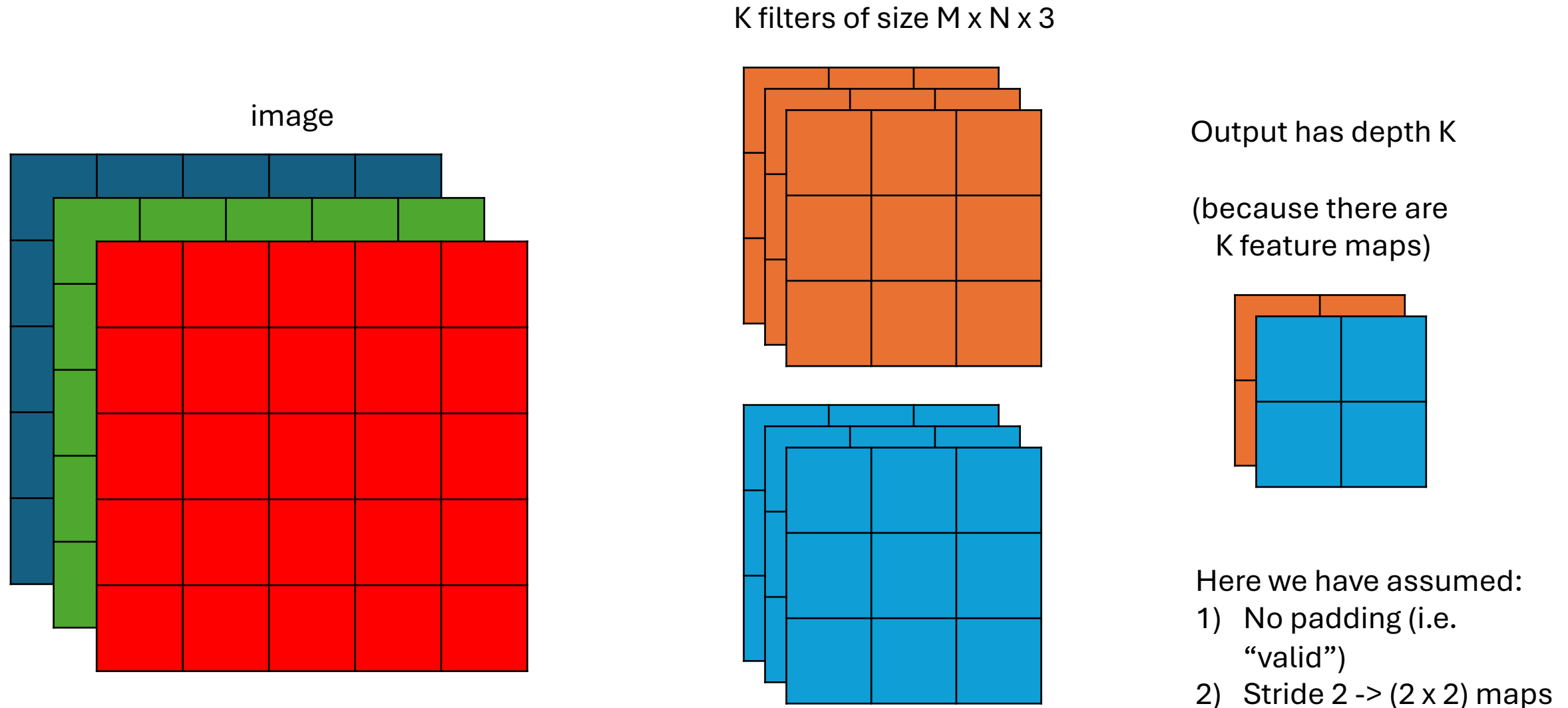
Important notes:

- 1) 3 key types of layers
- 2) Progressive decrease in spatial resolution
- 3) Progressive increase in number of features

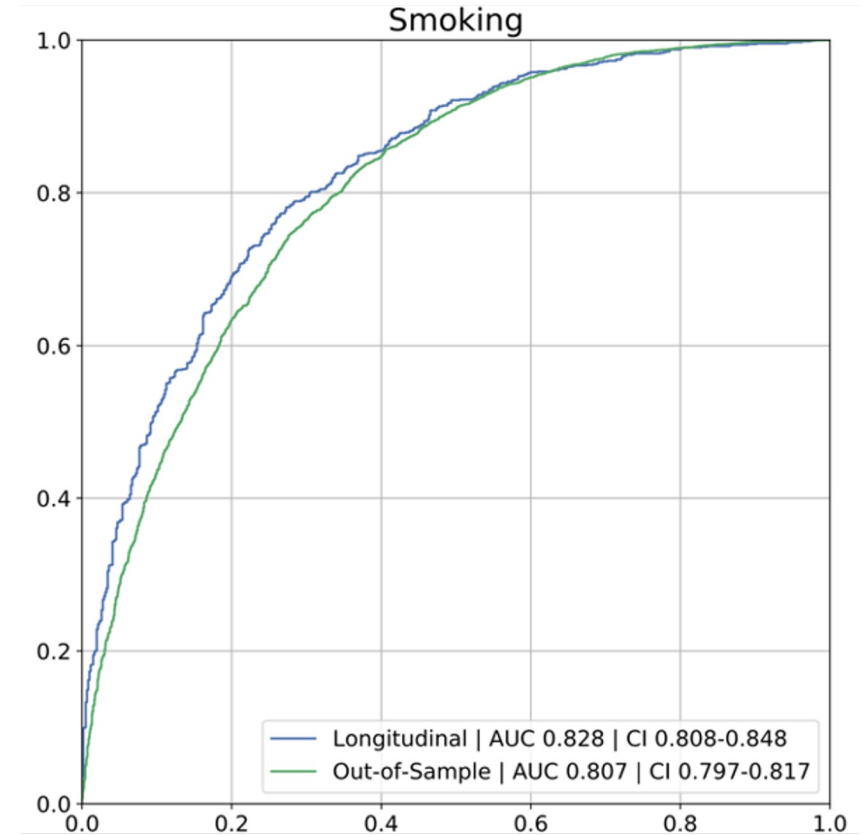
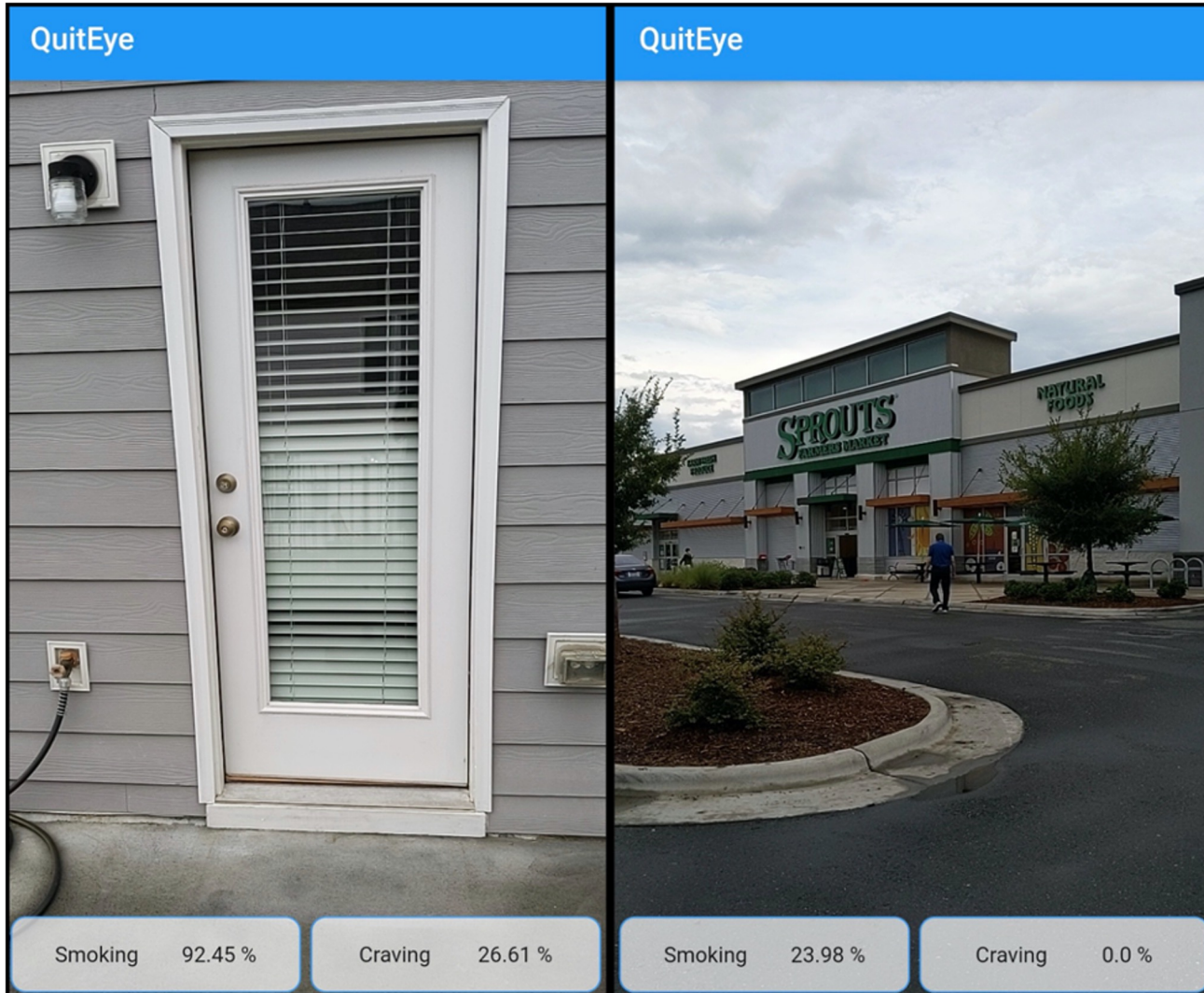
Parameter efficiency

- 1) PE architecture
- 2) PE fine-tuning

MobileNet: a parameter-efficient approach

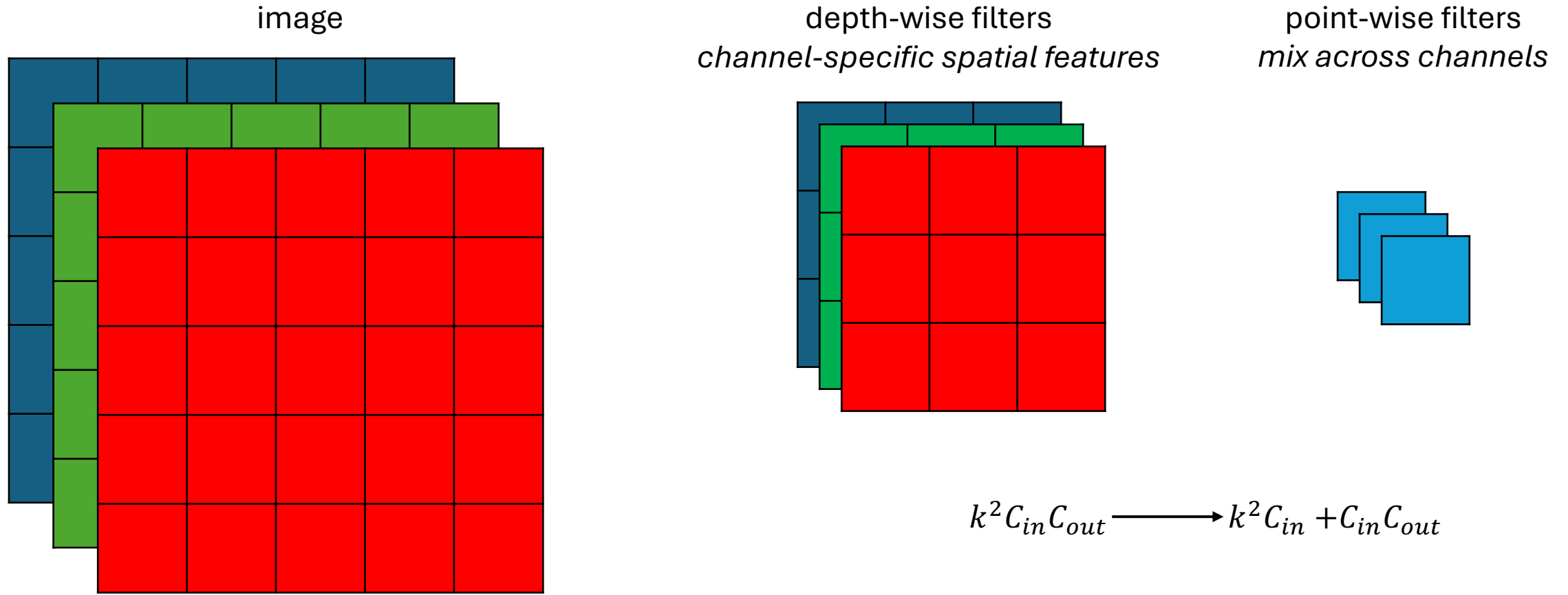


Predict smoking risk in real time via mobile device



Engelhard MM, D'Arcy J, Oliver JA, Kozink K, McClernon FJ. Machine Learning Prediction of Smoking Risk from Repeated Sampling of Environmental Images. *JMIR*.

“Depthwise separable” convolutions



*sometimes we learn m channel-specific spatial features

LoRA: low-rank adaptation of LLMs

- The original paper applied this to weight matrices in self-attention modules
- Pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$
- $W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ for $r \ll \min(d, k)$
- Set r , then update B and A during training

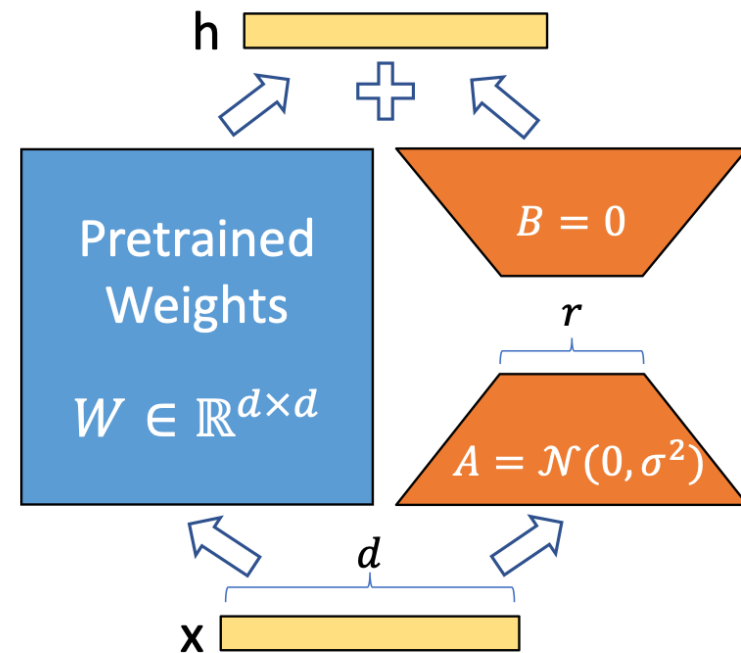


Figure 1: Our reparametrization. We only train A and B .

Low-rank adaptation of CNNs

- Consider conv layer weights
 $W_0 \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$
- Reshape to
 $W_0 \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} k^2)}$
- $W = W_0 + BA$, where $B \in \mathbb{R}^{C_{\text{out}} \times r}$ and $A \in \mathbb{R}^{r \times (C_{\text{in}} k^2)}$ for $r \ll \min(C_{\text{out}}, (C_{\text{in}} k^2))$
- Set r , then update B and A during training

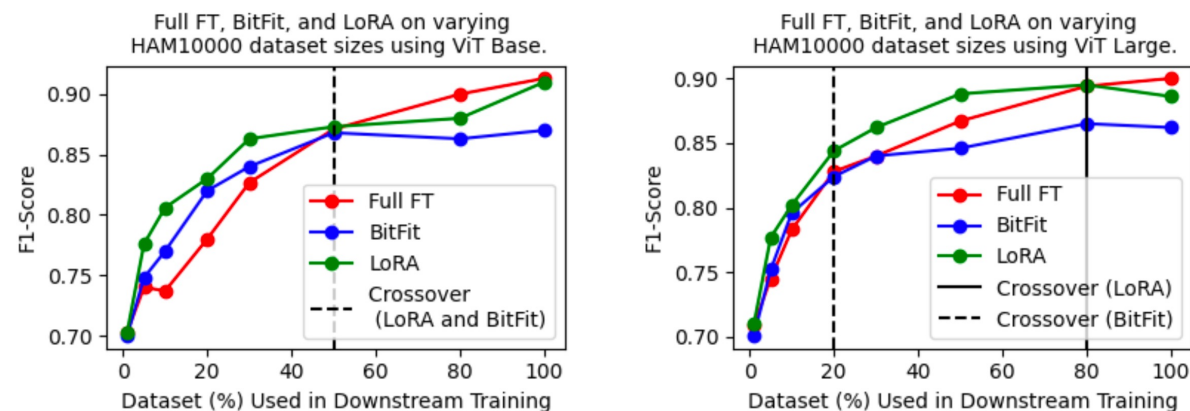


Figure 1: Plots showing the performance comparison for Full Fine-tuning, BitFit and LoRA with varying downstream dataset size for ViT-Base and ViT-Large models.

Parameter-Efficient Fine-Tuning for Medical Image Analysis: The Missed Opportunity. Raman Dutt, Linus Ericsson, Pedro Sanchez, Sotirios A. Tsaftaris, Timothy Hospedales. Proceedings of The 7th International Conference on Medical Imaging with Deep Learning, PMLR 250:406-425, 2024.

Residual connections

- $\text{output} = x + F(x)$
- Intuition: gradually refine the representation rather than starting from scratch each time
- ResNets remain state of the art; a common starting point for many of your models

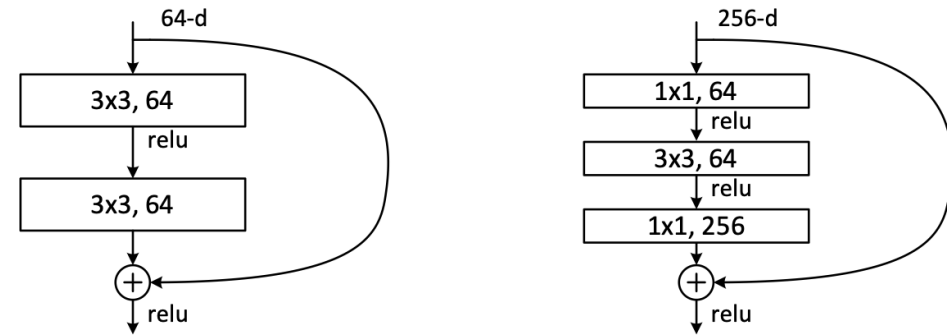


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition 2016 (pp. 770-778).

Residual connections

- $\text{output} = x + F(x)$
- $d/dx (x + F(x)) = 1 + F'(x)$
- Avoids vanishing gradients in earlier layers

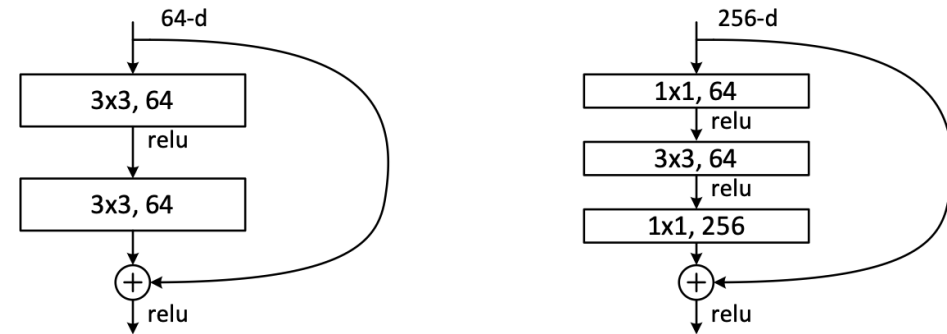
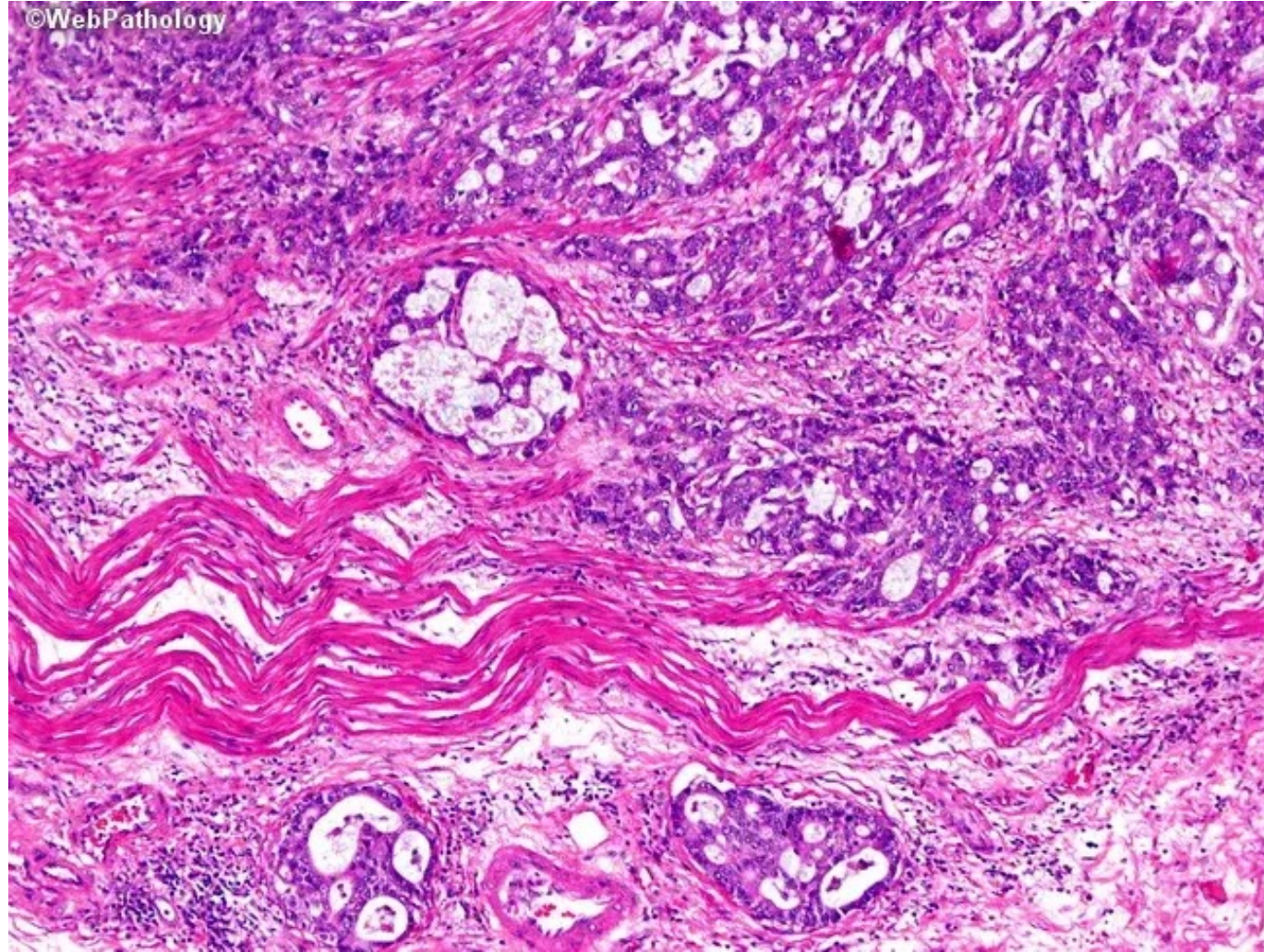
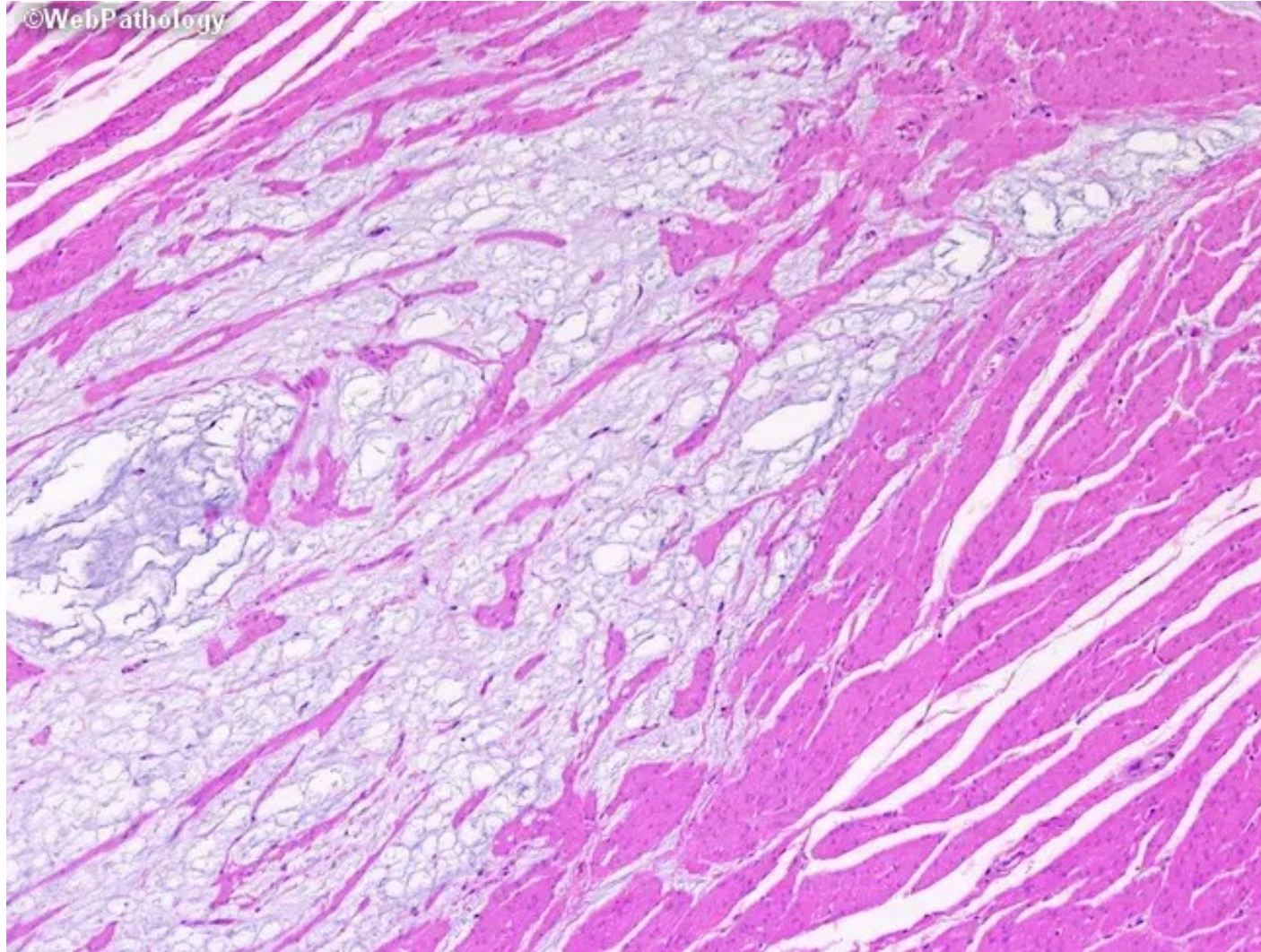


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

Example: pathology



In this example, the tumor has extended beyond muscularis mucosa to invade the submucosa (infiltrating glands at the bottom). Most cases of adenocarcinoma in Barrett esophagus are quite advanced at the time of diagnosis, often with full-thickness invasion of the wall and lymph node metastases.



Prominent pools of dissecting mucin in the walls of esophagus. Tumor cells are not visible. The patient had invasive adenocarcinoma that arose in a background of Barrett esophagus.

Unique characteristics

1. Images are extremely high resolution
2. Important findings tend to be localized to small regions
3. However, typically the whole slide is labeled (e.g., malignant, benign)

Approach: multi-instance learning (MIL)

Typical approach:

- Divide the image into patches
- Apply CNN encoder to each patch
- Aggregate patch embeddings
- Predict from aggregated representation

Aggregation methods:

- Max pooling (if any patch is cancerous, the whole thing is cancerous)
- Mean pooling
- Learned (i.e., attention-based) pooling

More efficient approaches

- “Hierarchical attention sampling”
- Gradually zoom in on the region of interest

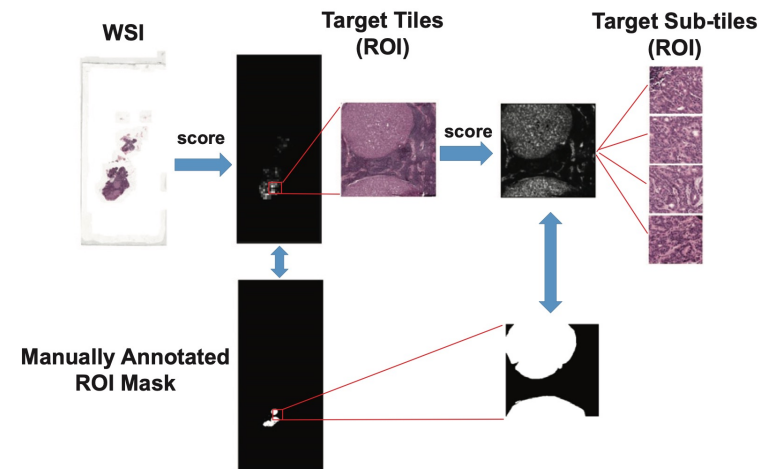


Figure 1. Illustration of processing a typical WSI using our zoom-in strategy. We see that *i*) there are large regions with little information (mostly background), and *ii*) small informative regions have high-resolution details. Leveraging the above characteristics of WSI, we derive a method that gradually zooms-in to the ROI. The proposed approach first process the down-sampled WSI to sample the target tiles, and then repeats this procedure to sample target sub-tiles. The sampled sub-tiles contain the fine-grained information for classification. The bottom images show that the manually annotated ROIs are captured by the proposed approach without the need for pixel level annotations.

More efficient approaches

- Learn to retrieve information most relevant to the classification task, and ignore everything else
- Designed for long documents but applies to large images

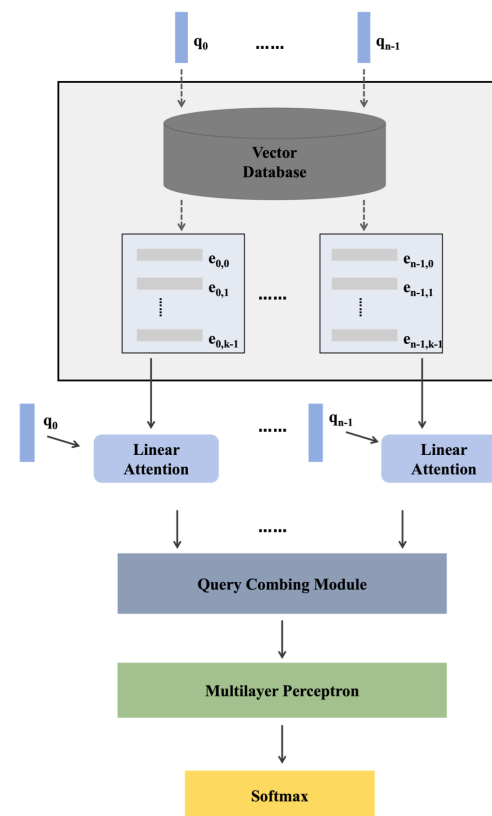


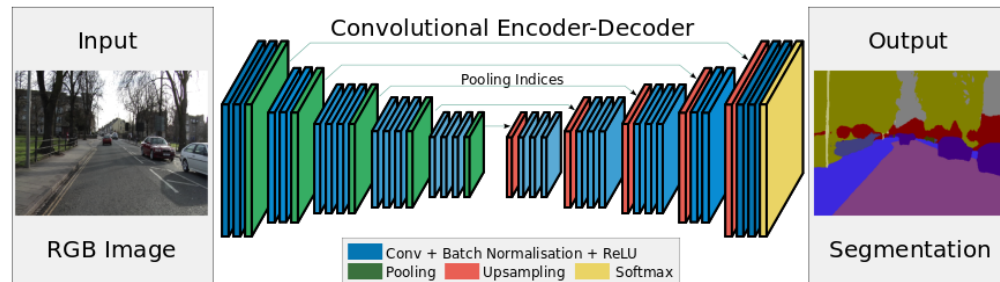
Figure 1: IRIS Model Architecture. The model has n query vectors set as learnable parameters, each responsible for retrieving k chunk embeddings from the vector database. Retrieved embeddings are aggregated through a linear attention mechanism. The dotted arrows in the figure correspond to the retrieval process that is not part of the model's computational graph.

Segmentation architectures

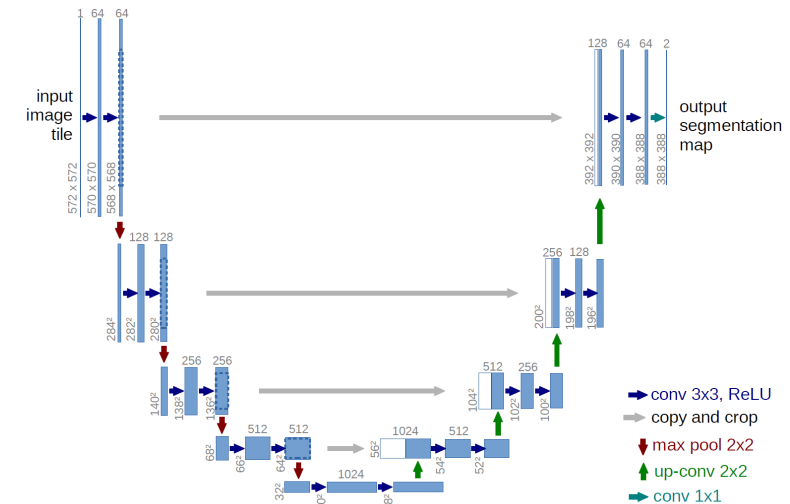
Two different approaches to spatial upsampling

Two U-shaped architectures

- Similar to other CNNs, we have progressive loss of spatial resolution
- Unlike CNNs, our goal is pixel-wise classification, so we then need to upsample

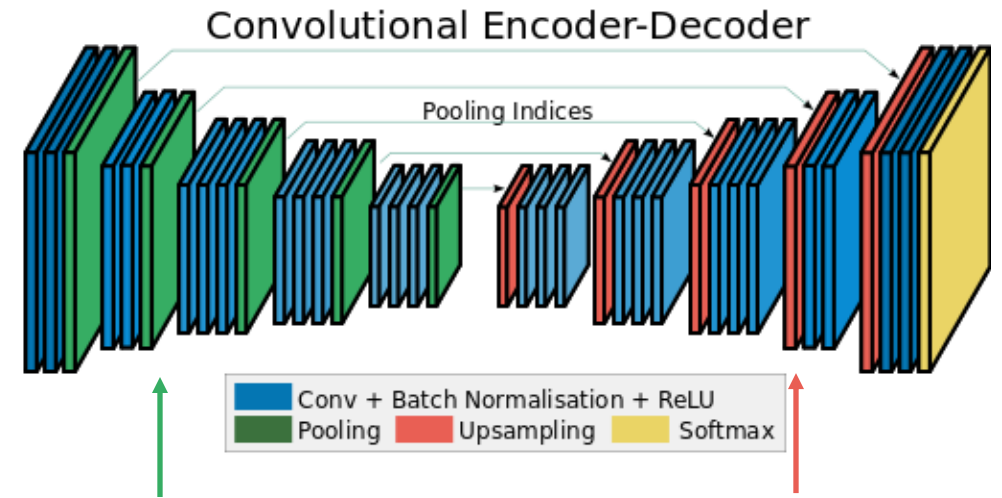


SegNet



U-Net

SegNet Decoder Details



Remember max pooling indices

3	1	3	7
2	5	3	4
1	4	3	1
1	2	2	5



5	7
4	5



Other
Layers



3	4
8	2



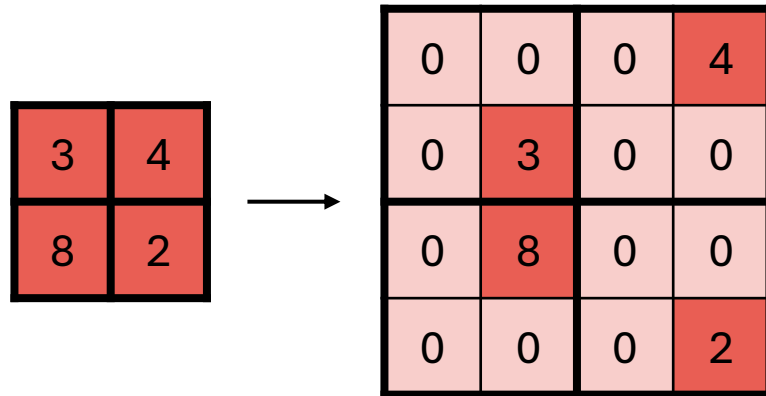
0	0	0	4
0	3	0	0
0	8	0	0
0	0	0	2



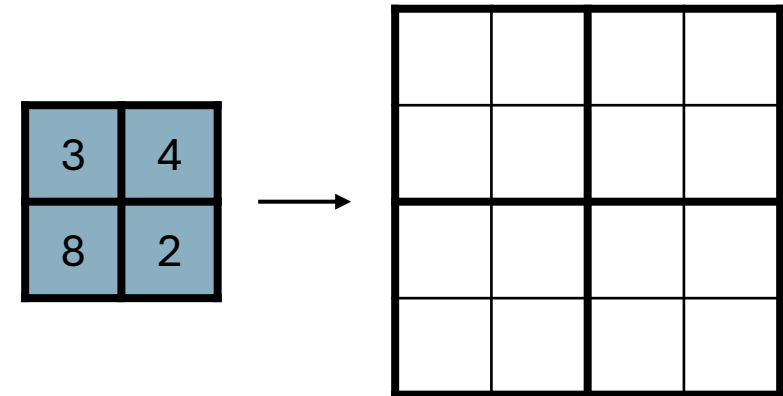
Convolution
with
trainable
decoder
filters

Use max pooling indices

SegNet: Upsampling w/ Stored Pooling Indices



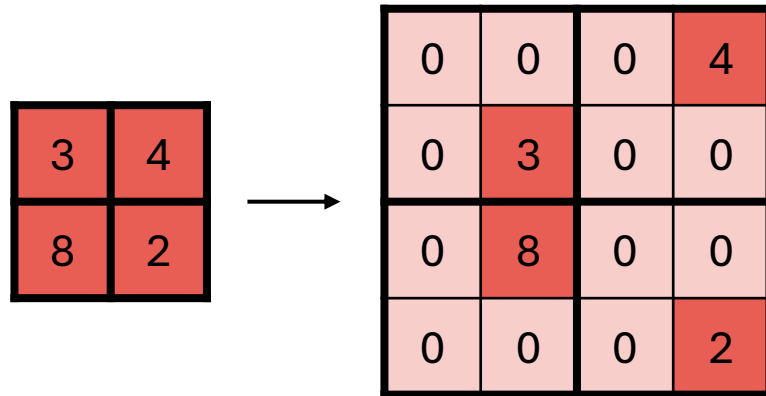
U-Net: learned upconvolutions



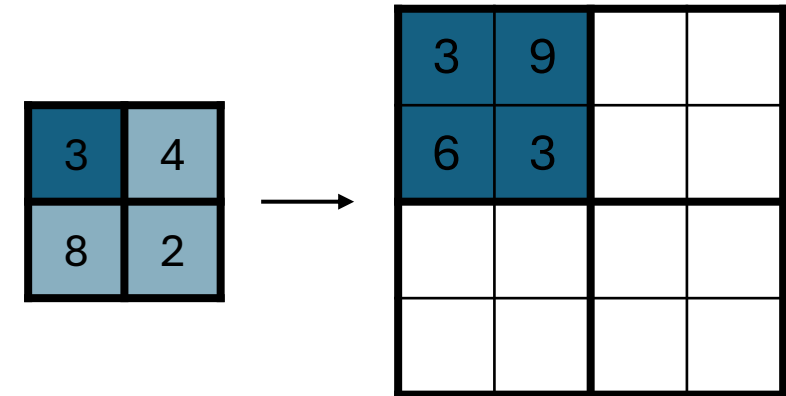
Learned filter

1	3
2	1

SegNet: Upsampling w/ Stored Pooling Indices



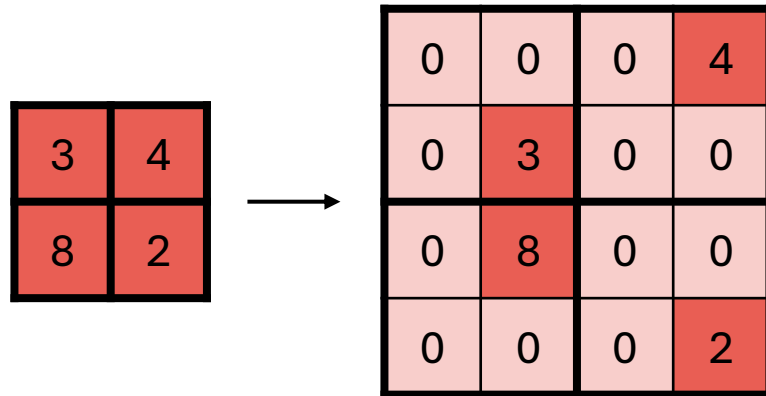
U-Net: learned upconvolutions



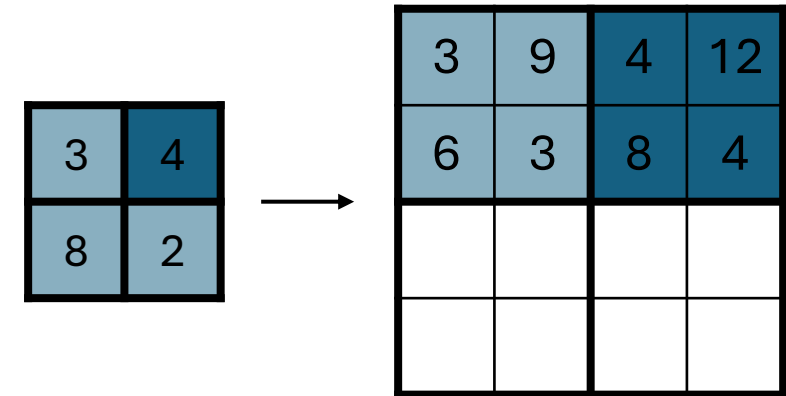
Learned filter

1	3
2	1

SegNet: Upsampling w/ Stored Pooling Indices



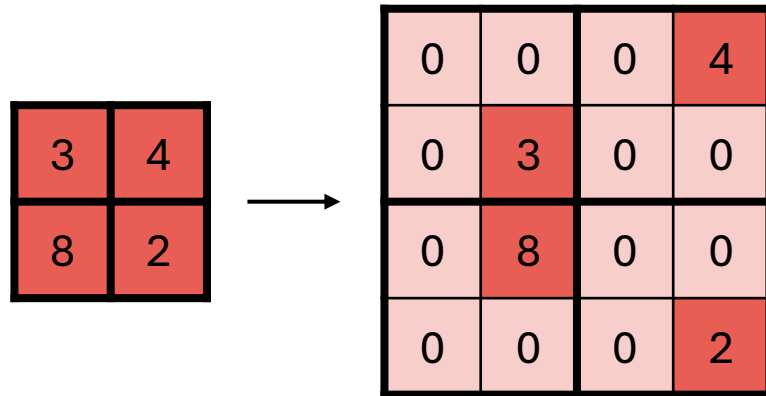
U-Net: learned upconvolutions



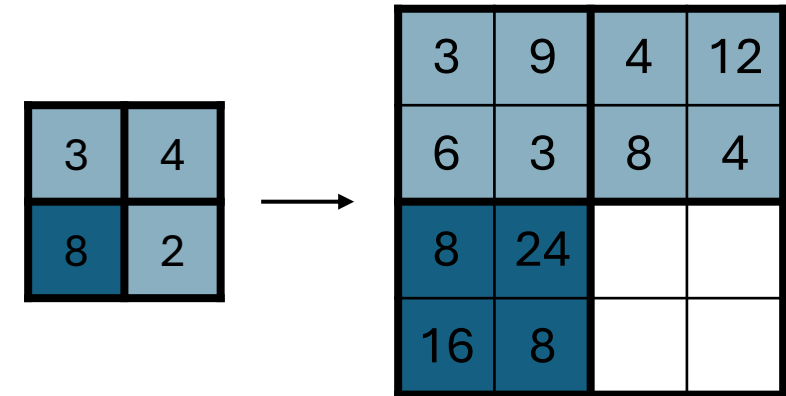
Learned filter

1	3
2	1

SegNet: Upsampling w/ Stored Pooling Indices



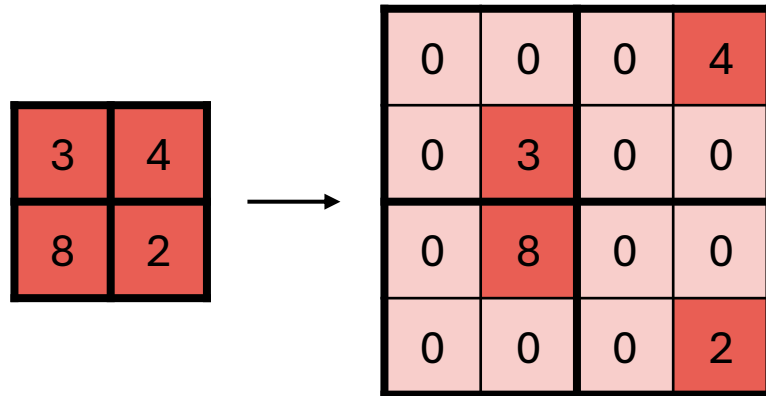
U-Net: learned upconvolutions



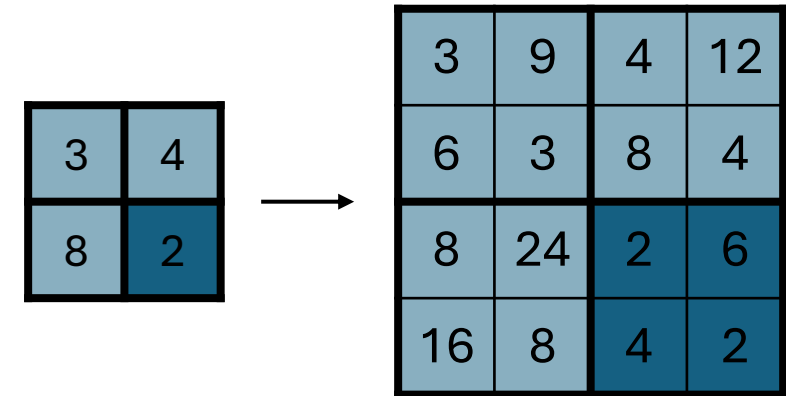
Learned filter

1	3
2	1

SegNet: Upsampling w/ Stored Pooling Indices



U-Net: learned upconvolutions



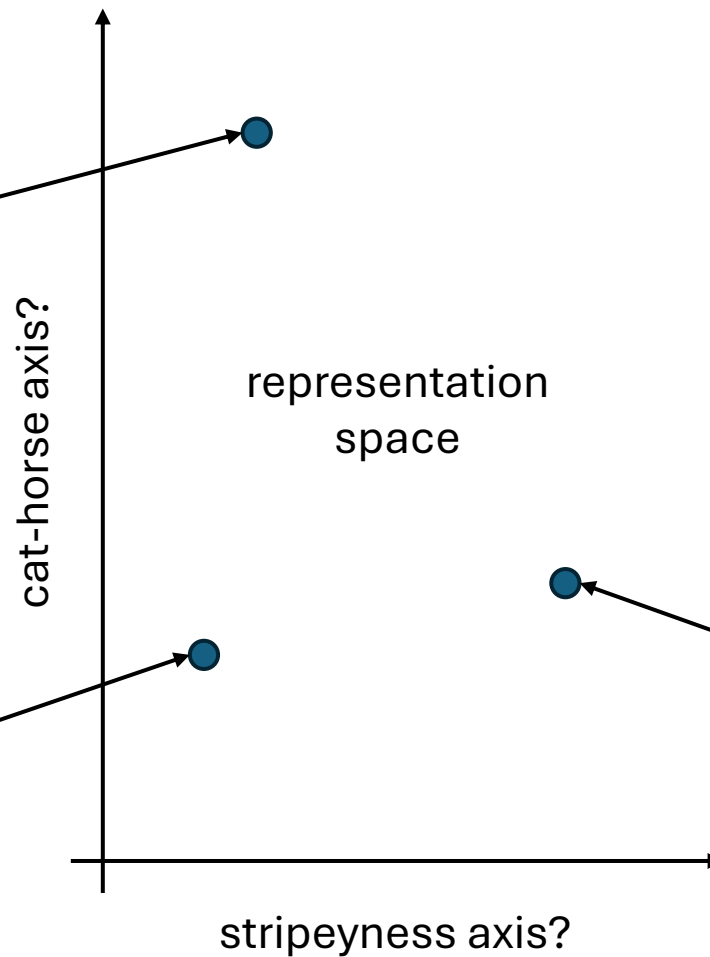
Learned filter

1	3
2	1

Zero-shot prediction



??



Report back (time permitting)

Calculate the size of the resulting feature maps as a function of:

- Input dimensions
- Padding scheme (valid, same)
- Stride

Report back (time permitting)

Demonstrate translation invariance in MNIST

- Digits appear only on the left-hand side in the training set
- Only on the right-hand side in the test set
- Train a shallow CNN
- Show translation invariance

Summary

- Transfer learning overview
- Parameter efficiency
- Residual connections
- MIL for pathology & elsewhere
- Segmentation basics
- Toward zero-shot learning